

iOS Client — Deep Technical Reference

Written for: you, as the engineer who owns this code. This document walks the entire iOS client **type by type, property by property, function by function**, and — just as importantly — says *why each is done that way* and *how it should be done* if you rewrite or extend it. It is the companion to [DEVELOPER GUIDE.md](#); that one explains the platform and build/sign/deploy, this one explains the code.

Source: [ios/Sources/LocationTrackerClient/](#).

0. Mental model before the details

The client is a small state machine feeding a durable pipe:

```
CoreLocation —fix→ LocationTracker —LocationPoint→  
CoreMotion —activity→ (decides accuracy,  
                        heartbeat, geofence)
```

Four design rules run through every file. When you extend the code, keep them:

1. **Durability over cleverness.** A location point is written to disk before anything else can go wrong. Nothing is deleted until the server acknowledges it.
2. **The OS is in charge of lifetime.** The app cannot decide to "run forever." It arranges to be *relaunched* and to *resume correctly*, including from a cold start with no UI.
3. **Everything user-facing state runs on the main actor.** CoreLocation/CoreMotion callbacks arrive on arbitrary threads

and are hopped onto `@MainActor` immediately.

4. **Assume every read can fail.** Keychain locked, disk locked, network down, token expired — each has an explicit branch, never a force-unwrap.

1. `AppConfig.swift` — the single source of tunables

`enum AppConfig` (a namespace; it has no cases, only `static lets`). Putting every constant here means behaviour is tuned in one file, not hunted across the code.

Constant	Value	What goes on
<code>defaultServerURL</code>	<code>https://34.199.20.93</code>	Server URL until closed on the screen
<code>pinnedCertificateSHA256</code>	<code>Set<String> of 2</code>	Which signed certificates are trusted
<code>uploadInterval</code>	<code>30 s</code>	Batch upload cadence moving
<code>lowPowerUploadInterval</code>	<code>60 s</code>	Batch upload cadence Low Power Mode
<code>tickInterval</code>	<code>30 s</code>	Housekeeping timer

heartbeatInterval	120 s	How long the device stays still before reporting location forces
stationaryAfter	180 s	No motion detected → low-power mode GPS
stationaryAfterMotionStill	60 s	Motion detected says "stationary" low-power mode soon
movingSpeedMps	1.0	Speed threshold count as moving
stationaryDistanceFilterMeters	50	Distance threshold while stationary
wakeFenceRadiusMeters	150	Geofence radius radius of area relaunch
watchdogDelay	1200 s (20 min)	Delay before "track" mode stops nudge
maxBatchSize	500	Points per batch upload request
maxQueuedPoints	20000	Cap on number of backlog points
maxAcceptedAccuracyMeters	150	Worst accuracy worth reporting

distanceFilterMeters	10
----------------------	----

Min
move
between
while

`enum ServerSettings` wraps the one value the user can change at runtime (`serverURL`) in `UserDefaults`, with `defaultServerURL` as the fallback. **How it should be done:** anything the *user* changes lives in `UserDefaults`; anything the *developer* sets lives as a static `let`. Don't mix the two.

2. `Models.swift` — DTOs and JSON

Plain `Codable` structs mirroring the server's JSON. Because the server emits camelCase matching Swift names, no `CodingKeys` are needed.

- **Request bodies:** `LoginRequest`, `RegisterRequest`, `LocationBatchRequest`.
- **The core datum:** `LocationPoint` — `latitude`, `longitude`, `accuracyMeters?`, `speed?`, `heading?`, `recordedAtUtc`. It is `Codable` (persisted to disk) **and** `Equatable` (dedup on reload after unlock).
- **Responses:** `AuthResponse`, `UserProfile`, `BatchIngestResponse` (`accepted`, `rejected`, `activeTripId`), `Trip`, `TripDetail`, `PagedResult<Item>`, `ServerErrorBody`.

`enum JSON` holds one shared `decoder` and `encoder`. The important part is the **date strategy**: .NET emits 0–7 fractional-second digits (`...:22Z`, `...:56.708553Z`). `ServerDate` normalises the fraction to exactly 3 digits before handing it to `ISO8601DateFormatter`, and appends `Z` if a timestamp lost its zone. **How it should be done:** never parse server dates ad hoc; route everything through `JSON.decoder` so this quirk is handled once.

3. `TokenStore.swift` — Keychain + the unlock probe

`enum TokenStore` stores `SessionTokens` (`accessToken`, `refreshToken`, `csrfToken`) in the Keychain.

- **base** — the query dict identifying the single generic-password item.
- **lock (`NSLock`)** — serialises access; the store is touched from background upload tasks and the main actor alike.
- **load () / save (_:) / clear ()** — the item is written with `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly`. That accessibility is deliberate: uploads must read the token while the phone is locked *in a pocket*, but the token must be unreadable before the first unlock after boot and must never sync off-device.
- **isUnlockedSinceBoot** — the crux of restart handling. It queries a dedicated probe item:
 - `errSecInteractionNotAllowed` → data is locked → **return false** (not yet unlocked).
 - `errSecItemNotFound` → first run → create the probe (only possible when unlocked) → true.
 - anything else → true. This is how the app distinguishes "my data is temporarily locked" from "my data is gone," which a plain `load () == nil` cannot.

How it should be done: secrets go in the Keychain, never `UserDefaults`; pick the *least* permissive accessibility that still lets background code work (here, after-first-unlock, this-device-only).

4. `SessionStore.swift` — who is signed in

`@MainActor final class SessionStore: ObservableObject`.
Drives which screen shows.

- **enum State: Equatable { restoring, signedOut,**

`signedIn(UserProfile) }`, published as `state`. The UI switches on it.

- `onSignedOut: ((_ explicit: Bool) -> Void)?` — the one hook that matters for correctness. `explicit == true` only when the user chose to sign out; `false` when the *server* expired the session. The app discards the offline backlog only on an explicit sign-out.
- `onSignedIn: ((UserProfile) -> Void)?` — fires on login and on restore; used to `claim` the queue for that user.
- `restore()` — at launch: if a token exists, calls `me()`; on success signs in, on 401 ends the session, on network failure keeps a cached profile so tracking continues offline.
- `login / register / logout` — thin wrappers over `APIClient`, updating `state`.
- `endSession(explicit:)` — clears tokens and cached profile, calls `onSignedOut`.
- A `sessionExpired NotificationCenter` observer bridges a server-side expiry (posted by `APIClient`) into `endSession(explicit: false)`.

How it should be done: keep auth *state* (`SessionStore`) separate from auth *transport* (`APIClient`). The store never touches `URLSession`; the client never touches `SwiftUI`.

5. `APIClient.swift` — networking, auth transport, TLS pinning

```
final class APIClient: NSObject, @unchecked Sendable,
a singleton (shared).
```

Structure

- `session: URLSession!` — assigned once in `init` and never mutated, which is what makes the `@unchecked Sendable` sound. The config disables the cookie jar entirely

(`httpCookieStorage = nil, httpShouldSetCookies = false`): the app manages the three auth cookies by hand rather than trusting `URLSession`'s `SameSite` handling for non-browser requests.

- **refresher: SingleFlight** — an `actor` at the bottom of the file that guarantees only one token refresh runs at a time (see below).
- **diagnosticsLock + _rejectedFingerprint** — records the fingerprint of a rejected cert so the login screen can show it; guarded by a lock because the TLS callback runs off-main.

The auth model (read this carefully)

The server issues `access_token`, `refresh_token`, `csrf_token` as **HttpOnly cookies**. The client:

- lifts them out of `Set-Cookie (storeSession(from:))` into the Keychain;
- on normal calls sends **only** `Authorization: Bearer <access>` — a bearer request carries no cookie, so the server's CSRF check doesn't apply;
- on **refresh/logout** sends the refresh + csrf cookies *and* the matching `X-CSRF-Token` header (the `.sessionCookies` credential mode).

Functions

- **login / register** — POST with `.none` credentials, store the returned session, decode a `UserProfile`.
- **me / myTrips / myTrip(id:) / myActiveTrip / uploadBatch** — go through `authorized`.
- **authorized(_:_:body:)** — sends with the bearer token; on **401** it refreshes **once** and retries. This one-retry rule prevents infinite loops.
- **refreshSession()** — wrapped in `SingleFlight`. Refresh tokens **rotate** on every use and the server treats reuse of a spent one as theft (revoking the whole chain), so two concurrent 401s must share one refresh, never race. On refresh failure it clears the

session and posts `sessionExpired`.

- **`perform(...)`** — builds the `URLRequest`, applies the chosen credential mode, returns `(Data, HTTPURLResponse)`. Rejects a non-https URL early.
- **`check(_:):`** — maps status codes to typed `APIErrors` (401 → `unauthorized`, 429 → `rateLimited` with `Retry-After`, 4xx/5xx → `server(status,message)`).
- **`storeSession(from:)`** — parses `Set-Cookie` and saves the three tokens.

TLS pinning (`urlSession(_:didReceive:)`)

The server uses a self-signed cert on a bare IP, so normal validation fails. Instead of turning validation off, the delegate:

1. lets a **publicly-trusted** cert through with default handling (so a real CA cert would just work);
2. otherwise computes the leaf cert's **SHA-256** and accepts it **only** if it is in `AppConfig.pinnedCertificateSHA256`;
3. records the fingerprint and cancels otherwise.

SingleFlight actor — `run(_:)` funnels concurrent callers onto one in-flight `Task`.

How it should be done: never disable TLS validation to accept a self-signed cert — pin it. Keep auth transport typed (`APIError`) so callers branch on meaning, not on status numbers. Make refresh single-flight whenever tokens rotate.

6. `LocationTracker.swift` — the heart

`@MainActor final class LocationTracker: NSObject, ObservableObject`. This is the biggest file; here is every member.

`enum MotionState`

unknown, stationary, onFoot, cycling, automotive, with isMoving and a display label. The motion coprocessor (M-series chip) reports this within seconds at negligible battery cost — far faster and cheaper than inferring motion from GPS positions.

Published state (drives the UI)

- authorization: CLAuthorizationStatus
- isTracking: Bool — are updates running now
- isStationary: Bool — in low-power mode
- motionState: MotionState
- lastLocation: CLLocation?
- lastError: String?

Private members

- manager: CLLocationManager, motion: CMMotionActivityManager — the two OS services.
- queue: UploadQueue — injected, not created here (testability, single owner).
- tickTimer: Timer? — the housekeeping timer.
- backgroundSession: CLBackgroundActivitySession? — the iOS 17 "I'm using location in the background" grant; recreated on each beginUpdates().
- powerObserver — watches Low Power Mode changes.
- wakeFenceCenter: CLLocation? — where the geofence currently sits.
- wakeFenceId — nonisolated fileprivate static so the delegate can compare it off-main.
- lastFlushAttempt, lastMovementAt, lastPointAt, heartbeatRequestedAt — timing bookkeeping.
- state: TrackerState? — tracking intent (see §6.7); nil while locked after boot.

6.1 The wantsTracking / pausedByUser computed pair

Both read/write through state. Two subtleties:

- `wantsTracking` **falls back to authorization == .authorizedAlways** when `state` is `nil` (locked after boot). Rationale: if the user granted Always, they want tracking; start on that guess and correct at unlock.
- `updateState(_:)` refuses to write when `state == nil` — you can't persist before the first unlock, and the saved state read at unlock is the authority anyway.

6.2 `reloadAfterUnlock()`

Called from the app's `protectedDataDidBecomeAvailable` observer. If `state` was `nil` and can now be read: adopt it; if the real intent is "off" but we started on the Always guess, stop; otherwise `resumeIfNeeded()`.

6.3 Lifecycle: `init`, `start`, `autoStart`, `pauseByUser`, `stop`, `resumeIfNeeded`

- **`init(queue:)`** — sets delegate, `activityType = .other`, and crucially `pausesLocationUpdatesAutomatically = false` (auto-pause would suspend the app and kill the heartbeat; power saving is done by lowering accuracy instead). Registers the power observer.
- **`autoStart()`** — starts unless the user paused it, it's already running, or denied. Called from the tracking screen's `onAppear`.
- **`start()`** — clears `pausedByUser`, sets `wantsTracking`, requests notification permission, then requests When-In-Use if undetermined (Always is only offered *after* When-In-Use).
- **`pauseByUser()`** — user's explicit off; stops and notifies. This is the only thing that keeps `autoStart` from restarting.
- **`stop(flushRemaining:)`** — tears down every service (updates, significant-change, visits, motion, geofence, background session, timer), disarms the watchdog. `flushRemaining == false` on sign-out because the queue is about to be cleared.
- **`resumeIfNeeded()`** — the cold-start entry point: if intent is on and authorised, begin.

6.4 beginUpdates () — turning everything on

Creates the background session; sets

`allowsBackgroundLocationUpdates = true` (this line **crashes without `UIBackgroundModes=location`** in Info.plist — a deliberate tripwire); starts standard updates, significant-change, visits, motion; arms the watchdog and posts "tracking on"; starts the tick timer; and upgrades to Always if only When-In-Use was granted.

6.5 Power modes: `applyMode(stationary:)`, `uploadInterval`

- **Stationary** → `kCLLocationAccuracyHundredMeters`, 50 m filter (Wi-Fi/cell, GPS mostly off).
- **Moving** → accuracy depends on `motionState`: driving gets `kCLLocationAccuracyBestForNavigation + .automotiveNavigation` (lets iOS snap to roads); walking/cycling get `.fitness`; unknown gets best. Low Power Mode drops each one notch.
- `uploadInterval` returns the low-power interval when the OS is in Low Power Mode.

6.6 The wake fence (§ region monitoring)

- `placeWakeFenceIfNeeded(at:)` re-centres the 150 m circle only after the user has moved $\geq 2/3$ of the radius, so a stationary phone doesn't churn it.
- `wakeFenceExited()` (from `didExitRegion`) marks movement and resumes.
- `removeWakeFence()` on stop.
- The calls go through a small `WakeFenceMonitoring` protocol implemented by `RegionMonitoring`, which uses the **classic `CLLocationManager` region API**. This indirection exists because the newer `CLMonitor` **crashed the app at launch on device**; the protocol is the single seam to swap implementations if you ever retry `CLMonitor`.

6.7 Motion: `startMotionUpdates`, `handleMotion`

Subscribes to activity updates; ignores `.low` confidence (they flip-flop); maps to `MotionState`; on a change, records movement and re-applies the mode (activity type and accuracy depend on *how* you move, not just whether).

6.8 The tick: `tick()`

Every `tickInterval`:

1. If the motion chip says moving, keep `lastMovementAt` fresh (so a traffic jam or train doesn't drop you to low power).
2. If still long enough (`stationaryAfter`, or `stationaryAfterMotionStill` when the chip agrees), switch to low-power mode.
3. Heartbeat: if a heartbeat fix was requested but none arrived within 60 s, restore the filter; else if it's been `heartbeatInterval` since the last point, drop the distance filter so the next fix arrives within seconds (that fix re-arms the filter in `handle`).
4. Re-arm the watchdog.
5. Flush the queue if `uploadInterval` has elapsed.

6.9 Fixes: `handle(_:)`, `isMovement(_ : heartbeat:)`, `point(from:)`

- **`handle`** — per fix: decide if it's a heartbeat and whether it shows movement; convert to a `LocationPoint` (dropping fixes worse than `maxAcceptedAccuracyMeters`); enqueue; update movement state; re-centre the fence. Then flush immediately if stationary (each fix is precious) or on the batch cadence if moving.
- **`isMovement`** — trusts a GPS-reported `speed ≥ movingSpeedMps`; otherwise compares distance from the last fix, with different thresholds when stationary (a 50 m unrequested fix, or a 150 m heartbeat fix, counts) vs moving ($≥ 15$ m at $≥ 1$ m/s).
- **`point(from:)`** — validates ranges (negative accuracy/speed/course mean "unknown") and builds the DTO.

6.10 CLLocationManagerDelegate (all nonisolated, hop to main)

- `didChangeAuthorization` — the cold-start path too: iOS calls it as the manager is created, before any UI; if intent is on, `beginUpdates()` runs headless. Denial stops and notifies.
- `didUpdateLocations` → handle.
- `didExitRegion` → `wakeFenceExited` (guards on the fence id).
- `didVisit` → `resumeIfNeeded`.
- `didFailWithError` — ignores transient `locationUnknown`.

6.11 TrackerState (bottom of file)

`Codable`, `Equatable` with `wantsTracking`, `pausedByUser`, stored in `Application Support/tracker-state.json` with `.completeFileProtectionUntilFirstUserAuthentication`. `load()` returns `nil` when locked, otherwise reads the file or migrates old `UserDefaults` keys. **Why a file, not UserDefaults:** after a reboot `UserDefaults` may keep serving the empty values it saw while locked; a file can be re-read deterministically once unlocked.

How it should be done: treat "before first unlock after boot" as a real, testable state, not an edge case. Persist intent where you can re-read it. Always hop OS callbacks to the actor that owns the state.

7. UploadQueue.swift — durable, offline-first uploads

```
@MainActor final class UploadQueue: ObservableObject.
```

Published (for the status screen)

```
pendingCount, lastUploadAt, lastUploadSummary, lastError,
```

`activeTripId` (the last also persisted to `UserDefaults`, so a trip that ends after a relaunch is still noticed).

Private state

- `pending: [LocationPoint]` — the in-memory queue, mirrored to `pending-locations.json`.
- `isFlushing` — reentrancy guard.
- `notBefore` — back-off deadline after a 429.
- `diskLoaded` — **false when the file exists but is locked** (post-boot). While false, `persist` is a no-op so the in-memory list can't clobber the saved backlog.
- `pathMonitor: NWPathMonitor + wasOnline / offlineSince / offlineNotified` — connectivity.

Key functions

- **`init`** — loads from disk (`loadFromDisk` returns `nil` if locked, `[]` if truly absent), restores `activeTripId`, and starts the path monitor. On **`reconnect`** it flushes immediately and fires "Back online"; on a drop longer than `offlineNoticeDelay` (60 s) it fires "Offline".
- **`reloadAfterUnlock()`** — after the first post-boot unlock: merge on-disk + in-memory points, sort by time, dedup (`LocationPoint: Equatable`), `persist`, `flush`.
- **`claim(for:)`** — drops another user's backlog so histories never mix.
- **`enqueue(_:)`** — append, trim to `maxQueuedPoints` (oldest dropped), `persist`.
- **`flush()`** — the core. Guards on not-already-flushing, non-empty, token present, back-off. Wraps the whole run in `beginBackgroundTask` so a batch in flight can finish before iOS suspends the app. Sends oldest-first `maxBatchSize` slices; on success drops the sent count and updates `activeTripId` (a change means a trip ended → `tripEnded`); on **429** sets `notBefore` and stops; on **4xx** drops the batch (it will never be accepted, and would block everything behind it); on anything else (`offline/5xx`) keeps the points and returns.

- `tripEnded(_:)` — fetches the now-final, smoothed trip and fires "Trip recorded."
- `persist()` — writes the file with complete-until-first-unlock protection; no-op while `!diskLoaded`.

How it should be done: write before you send; delete only on ack; distinguish *retryable* (offline/5xx/429) from *permanent* (4xx) failures; and never let one poison batch block the queue.

8. `Notifier.swift` & `TrackingWatchdog.swift` — local notifications

- **Notifier** posts trip summaries, tracking on/off, and offline/online. Each category reuses a **fixed identifier** so a newer notification *replaces* the older one (no stacking). Trip ids are remembered so each trip notifies once. `showsInForeground` lets trip/connectivity banners appear while the app is open, but suppresses "sharing is on" (redundant on-screen).
`NotificationPresenter` is the `UNUserNotificationCenterDelegate` implementing that.
- **TrackingWatchdog** is a dead-man's switch: it keeps one notification scheduled `watchdogDelay` (20 min) in the future and pushes it back on every `arm()`. If the app stops running, nothing pushes it back and it fires ("tap to resume"). This works even though iOS does **not** deliver a termination callback when an app is swiped away.

How it should be done: reuse identifiers to avoid notification spam; use a self-rescheduling timer as a liveness signal rather than relying on a termination callback that may never come.

9. `LocationTrackerClientApp.swift` —

composition root

```
@main struct LocationTrackerClientApp: App.
```

- **init** creates the three objects, wires `onSignedOut` (stop tracking; clear the queue only on explicit sign-out) and `onSignedIn` (`queue.claim`), sets the notification delegate, and registers the **protectedDataDidBecomeAvailable** observer that calls `tracker.reloadAfterUnlock()` and `queue.reloadAfterUnlock()` — the whole post-reboot recovery.
- **RootView** switches on `session.state` (restoring → spinner, signedOut → `LoginView`, signedIn → `TrackingView`), runs `session.restore()` in `.task`, and on becoming signed-in calls `tracker.resumeIfNeeded()`.

How it should be done: build all singletons once in the composition root and inject them via `environmentObject`; don't let views create services.

10. `LoginView.swift` & `TrackingView.swift` — UI

- **LoginView** — server URL (editable), email, password, and a register toggle. Normalises the URL (trims, strips trailing slash), maps `URLErrors` to friendly text, and shows the pinned fingerprint if the cert check failed.
- **TrackingView** — the share toggle (`start` / `pauseByUser`), a status section (GPS mode, motion state, queue depth, last upload, active trip, server), and the trip list, refreshed every 15 s and on pull-to-refresh. `onAppear` calls `tracker.autoStart()`.

How it should be done: keep views thin — they read published state and call intent methods (`start`, `pauseByUser`, `logout`); no business logic lives here.

11. Concurrency model (how the pieces stay thread-safe)

- `LocationTracker`, `UploadQueue`, `SessionStore` are all `@MainActor`: their mutable state is only ever touched on the main actor.
- `CoreLocation` and `CoreMotion` callbacks are declared `nonisolated` and immediately `Task { @MainActor in ... }` (or `MainActor.assumeIsolated` inside a callback already delivered to `.main`).
- `APIClient` is `@unchecked Sendable` — justified because its only stored state (`session`) is set once and never mutated, and diagnostics are lock-guarded.
- Token refresh is serialised by the `SingleFlight` actor.

How it should be done: pick one actor to own each piece of mutable state and funnel every OS callback onto it. Reach for `@unchecked Sendable` only with a written justification like the one above.

12. If you extend it — a checklist

- **New API call?** Add it to `APIClient` going through `authorized`, add its DTO to `Models`.
- **New persisted state?** If it's a secret → `Keychain`; if it must survive reboot-before-unlock → a protected file with an unlock-reload path; otherwise `UserDefaults`.
- **New background trigger?** Wire it through the existing `CLLocationManagerDelegate`; prefer long-established `CoreLocation` APIs over the newest ones on sideloaded builds.
- **New tunable?** Put it in `AppConfig`, never a literal in logic.
- **New user-facing string from server data?** Render with `textContent`-equivalent safety; never interpolate untrusted text into anything executable.
- **Touching lifetime/relaunch?** Re-read [DEVELOPER GUIDE.md](#)

[§3–4](#); test the force-quit and reboot scenarios on a real device, not the simulator.

Keep this file in step with the code: when you rename a function, move a constant, or change a threshold, update the matching section here.

13. The exact Swift/iOS commands that "turn the code on" — a guided read-through

This section walks the app **in the order the calls actually fire**, naming the exact framework API each line invokes and what it switches on. Read it with the source open; it is the "go through the code as if you wrote it" tour.

13.1 Cold launch → objects created

`@main struct LocationTrackerClientApp` is the process entry point. `SwiftUI` instantiates it and its `init` runs:

```
let session = SessionStore()
let queue   = UploadQueue()           // opens N
let tracker = LocationTracker(queue: queue) // creates
```

- `CLLocationManager()` allocates the location service object. **Nothing happens yet** — it only produces data after a `startUpdating...` call.
- `manager.delegate = self` registers this object to receive callbacks (`locationManager(_:didUpdateLocations:)`, etc.).
- `manager.pausesLocationUpdatesAutomatically = false` — **the switch that keeps the app alive while still**. Left `true`, iOS pauses updates when it thinks you stopped, suspends the app, and the heartbeat dies.
- `CMMotionActivityManager()` is the handle to the motion

coprocessor; inert until `startActivityUpdates`.

Then the app registers the reboot-recovery hook:

```
NotificationCenter.default.addObserver(  
    forName: UIApplication.protectedDataDidBecomeAvailable
```

`protectedDataDidBecomeAvailable` fires **the moment the user first unlocks after boot**, i.e. when the Keychain and protected files become readable.

13.2 Deciding the first screen

`RootView.body` switches on `session.state`; `.task { await session.restore() }` runs when the view appears. `restore()` calls `APIClient.me()`. If a token exists and works → `signedIn`.

13.3 Turning tracking ON — the precise call sequence

`TrackingView.onAppear` → `tracker.autoStart()` → `start()`.
On first run:

```
manager.requestWhenInUseAuthorization()    // shows the Wh
```

When the user answers, iOS calls

`locationManagerDidChangeAuthorization`. If authorised and intent is on, `beginUpdates()` runs — where every background capability is switched on, line by line:

```
backgroundSession = CLBackgroundActivitySession()    //  
manager.allowsBackgroundLocationUpdates = true      //  
manager.showsBackgroundLocationIndicator = true      //  
manager.startUpdatingLocation()                      //  
manager.startMonitoringSignificantLocationChanges()  //  
manager.startMonitoringVisits()                     //  
startMotionUpdates()                                //  
manager.requestAlwaysAuthorization()                 //
```

- **(1) `CLBackgroundActivitySession()`** — iOS 17's explicit background-location grant. Creating the object turns it on; `invalidate()` turns it off. This makes "keep tracking with the screen locked" legal without a visible app.
- **(2) `allowsBackgroundLocationUpdates = true`** — **crashes the process if `UIBackgroundModes` lacks `location`.** That crash is intentional: it forces Info.plist and code to agree.
- **(4) `startUpdatingLocation()`** — the standard, high-accuracy GPS stream. Fixes arrive via `didUpdateLocations`. Primary data source.
- **(5) `startMonitoringSignificantLocationChanges()`** — a separate, low-power service that **relaunches a terminated app**. Survives iOS killing the app; not force-quit.
- **(6) `startMonitoringVisits()`** — coarse "arrived/left" events; another relaunch path.
- **(7) `motion`:** `motion.startActivityUpdates(to: .main) { activity in ... }` — the coprocessor calls back with `CMMotionActivity(.automotive, .walking, .stationary, each with a confidence)`.

The geofence is armed from the first fix, in `placeWakeFenceIfNeeded`:

```
let region = CLCircularRegion(center: coord, radius: 150,
region.notifyOnExit = true
manager.startMonitoring(for: region)    // (9) survives te
```

- **(9) `startMonitoring(for:)`** — region monitoring. iOS watches the circle in its own daemon, so it relaunches the app on exit even after a reboot. Delivered via `didExitRegion`.

Housekeeping timer:

```
Timer.scheduledTimer(withTimeInterval: 30, repeats: true)
```

A `Timer` fires only while the app is actually running (foreground or an active background session); it is not itself a wake-up source — the location services above are.

13.4 A fix arrives → data flows

`didUpdateLocations` → hops to main → `handle(_):`

- reads `location.horizontalAccuracy`, `.coordinate.latitude/longitude`, `.speed`, `.course`, `.timestamp` — the raw `CLLocation` fields;
- builds a `LocationPoint`, calls `queue.enqueue(point)`, which does `data.write(to:options:[.atomic, .completeFileProtectionUntilFirstUserAuthentication])` — the command that persists the point **encrypted at rest**;
- re-centres the geofence.

13.5 Upload → network

`queue.flush():`

```
let bg = UIApplication.shared.beginBackgroundTask(withName:
... URLSession.data(for: request) ...
UIApplication.shared.endBackgroundTask(bg)
```

- **`beginBackgroundTask`** — asks iOS for extra seconds to finish work when backgrounded; without it, a suspend mid-request aborts the upload. Always paired with `endBackgroundTask` (in `defer`).
- **`URLSession.data(for:)`** — the actual network I/O over the pinned-cert session.

13.6 Turning tracking OFF

`stop()` calls the exact inverses:

```
manager.stopUpdatingLocation()
manager.stopMonitoringSignificantLocationChanges()
manager.stopMonitoringVisits()
motion.stopActivityUpdates()
manager.stopMonitoring(for: region)           // via removeWake
```

```
manager.allowsBackgroundLocationUpdates = false
backgroundSession?.invalidate()           // ends the iOS
tickTimer?.invalidate()
```

13.7 The relaunch path (no UI ever shown)

When iOS relaunches the app in the background (significant-change, geofence exit, visit, or the `voip` boot mode), SwiftUI still builds the `App`, so `init` runs and the objects are created. The `CLLocationManager` is created with the same delegate, and iOS immediately calls `locationManagerDidChangeAuthorization` — **before any view appears**. That handler calls `beginUpdates()` if intent is on. This is why tracking resumes with no screen: the entry point is the delegate callback, not the UI.

13.8 Info.plist keys that unlock these APIs

None of the above works without the matching declarations in [ios/Info.plist](#):

- `UIBackgroundModes = [location, voip]` — background delivery and the boot relaunch.
- `NSLocationWhenInUseUsageDescription`, `NSLocationAlwaysAndWhenInUseUsageDescription` — required, or the authorization requests are silently ignored.
- `NSMotionUsageDescription` — required, or `startActivityUpdates` fails.
- `NSAppTransportSecurity` → `NSAllowsArbitraryLoads` — lets the pinned self-signed HTTPS connection proceed (the app still enforces the pin itself).

How it should be done: every runtime capability on iOS is a *pair* — a Swift call **and** an Info.plist declaration. If a `startX` seems to do nothing, the missing half is almost always the plist key.